MagicCube

简介

- 背景
- 依赖版本
- 模块介绍

概要设计

- 问题分析
- 设计思路
- 架构设计

详细设计

- 规则设计
- 规则载体
  - Json文件
  - DB
- 组件设计
  - MagicEngine
  - MagicProcessor
  - MagicFilter
  - MagicStrategy
  - MagicHandler
  - 其它组件
- 流程设计
- 规则数据来源
  - MagicPlan来源
  - MagicCode来源

Strategy介绍

- 主要数据对象
  - MagicNode
  - ResultNode

- Rule的关键属性
- 接口说明
  - execute
  - doExecute
- 常用Strategy
  - valueCopy
  - fixedValue
  - codeMap
  - dateFormat
  - numberFormat
  - stringFormat
  - publishCacheEvent
  - consumeCacheEvent
  - groovyFormula
- 自定义Strategy
- Rule路径

测试

- 相关API
  - 核心API
  - 资源类API
- 测试
  - 测试数据
  - 测试Case
    - valueCopy使用
    - List条件拆分
    - 动态Rule Case
    - 字符串格式化Case
    - 数值格式化Case
    - codeMap Case
    - stepStrategy应用Case
    - KeyNameStategy Case
    - 缓存与公式计算Case

GlobalRule和公式计息Case

CacheEvent use Case

特别建议

简介

背景

MagicCube，魔方之意，设计初衷是让数据模型之间自由转换更容易、更简单。数据是企业的核心资产，这种核心资产也是有年龄的，在时间的轨迹上，数据模型也在成长变化中，甚至阶段性的变化，如何让数据模型在时间上兼容？在信息化社会中，数据不再是一个孤岛，是一片纵横交错的网，无处不存在着交换，如何用最小的代价让数据适配交换？即便在同一系统中，模型在不同的场景所需要的信息量也不同，如何快速的调整模型结构以适应各种场景？这里提供了一种解决方案。

依赖版本

版本	版本
springboot	3.5.6
JDK	21

模块介绍

magic-app

这是一个测试模块，用于起一个springboot应用测试magic的功能。

magic-core

这是Magic的核心模块。

magic-resource

Magic的数据模型模块，主要是数据DB的保存与查询。

docs

Magic相关文档。

概要设计

问题分析

数据模型间的转换没有太多技术难度，有初级编程经验并熟悉模型开发的都可以搞定，即便模型足够大，只要堆代码都可以搞得定，但是这是在不考虑后期维护性的前提下可行。在大量if-else堆积下，每一个新接手的开发要做的第一件就先骂人，然后边干边骂，因为后期每一个调整都需要在海量的if-else中找切入点，稍有不慎便是灾难现场。

复杂模型转换，乍看上去挺复杂的，其实也挺乱的，点太多，也很散乱。这时不妨静下心来，梳理一下转换过程，抽丝剥茧般逐步分解转换过程，还是有迹可寻的：

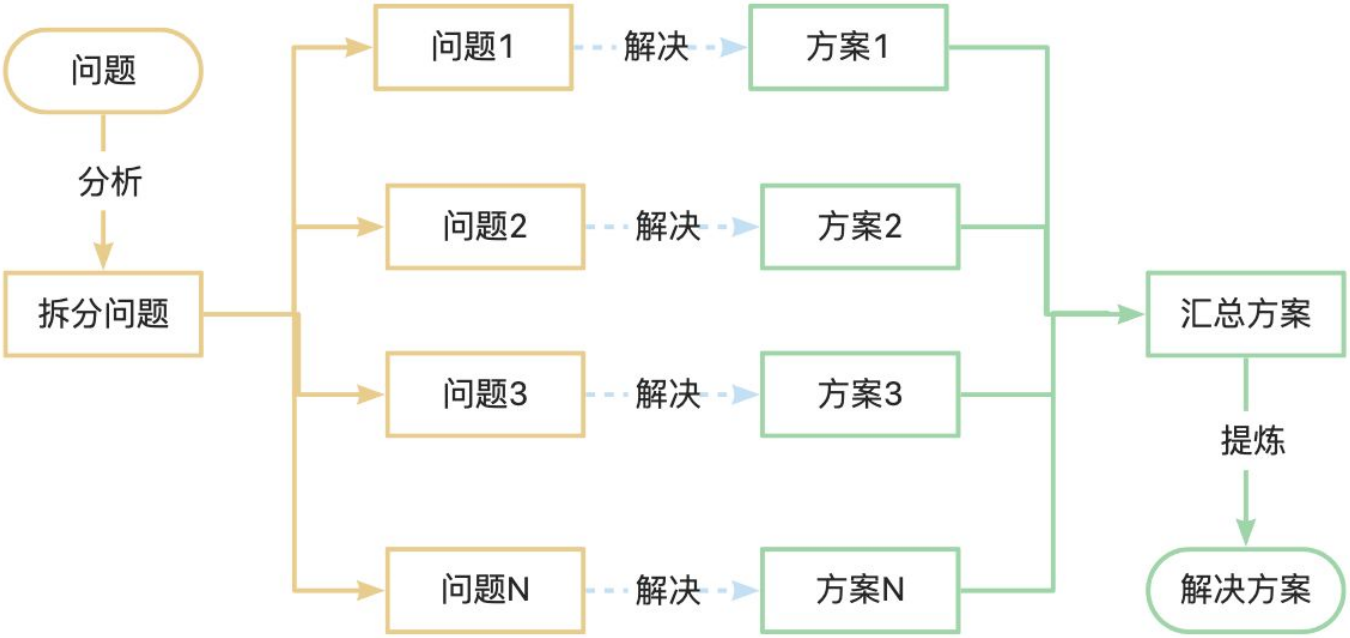
- 首先，模型转换时，数据源是已知的，转换目标也是已知的，未知的是中间过程。
- 其次，一把抓看问题，得到的只是一团乱麻，不妨换个角度，着眼于微末，模型是由各种属性通过组织构建而成，模型的最小信息单位是属性，模型转换也就是A模型的a_1属性转换为B模型的b_1属性，简言之，模型转换的本质是模型属性的转换。
- 最后，如果把属性的转换看做是一个动作，模型转换也就是所有相关模型属性转换动作的集合。通常模型越大，集合也越大，但是不要被这个集合大小吓到，若是对集合动作归类后会发现，只是有限的几类常用动作，偶尔会有几个特殊动作存在。到此，模型转换便简单到几个转换动作，再加个别特殊动作处理，清晰简单。

此时再回首，经过一系列的逐步分解归类后，模型最复杂的“量”没了，整体转换过程也简单明了了。

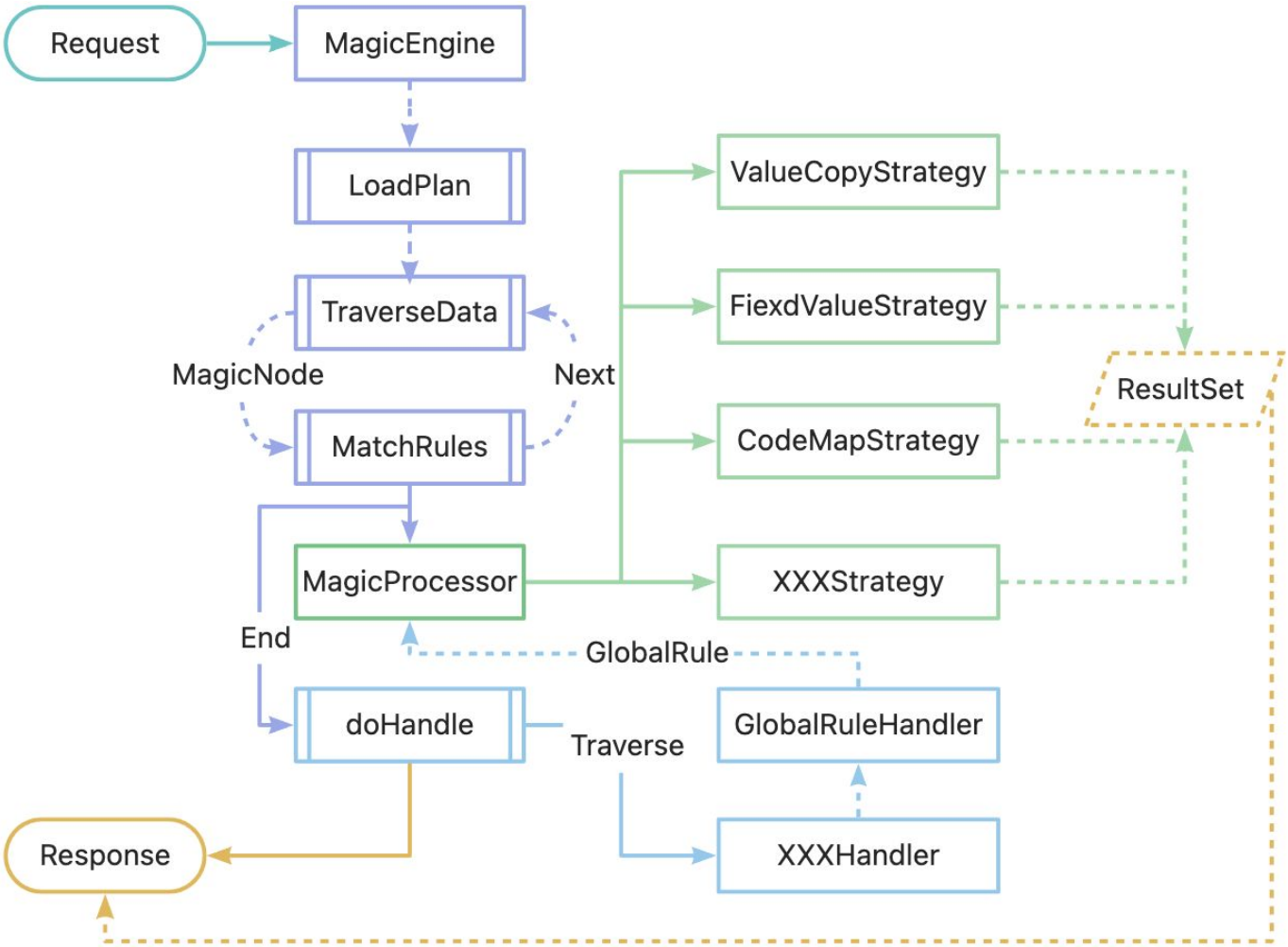
设计思路

一言概之，四个字：化繁为简。

处理复杂大问题时，当大于无处着手时，不妨换个角度，从细微处着手，或许有意外的收获。看似复杂的问题经过抽丝剥茧，逐步拆分成一个个小问题并解决，最后再把这些小问题的解决方案整合串联起来，复杂的问题也就解决了。一口吃不小时，硬吃容易噎到，不妨一小口一小口慢慢吃掉。



架构设计

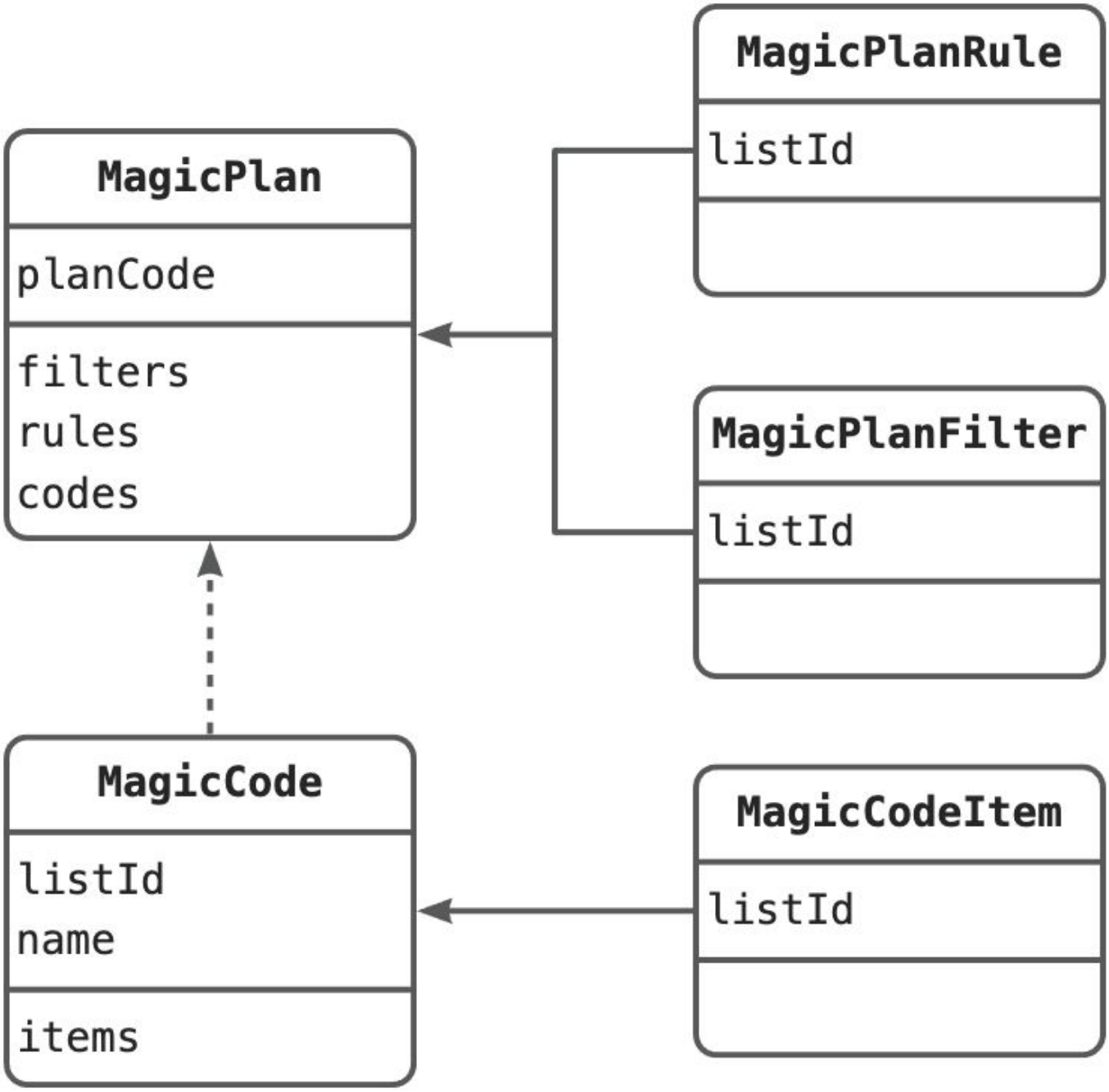


> 1. Request必须携带Data做为模型数据源，进入到MagicEngine中。

2. LoadPlan，根据PlanCode加载对应的MagicPlan（模型转换核心规则配置表），如果未匹配到MagicPlan，则会异常并中断执行，如果Request中带有，则优先使用。。
3. TraverseData，遍历Data数据，逐步转换为MagicNode树结构。
4. MatchRules，有两种Match：TraverseData遍历时，每个叶子节点的MagicNode会与Rule进行匹配；DynamicRule时，Rule主动与MagicNode匹配。
5. MagicProcessor，Node与Rule绑定后，中根据Rule中指定的strategy找到MagicStrategy进行转换操作，结果保存在上下文MagicContext的ResultSet中。
6. doHandle，考虑到某些特殊需求，故在最后增加了MagicHandler来处理一些特殊情况，GlobalRule便是其中一种特殊情况。
7. ResultSet中数据经过处理后，返回目标模型数据。

详细设计

规则设计



- MagicPlan

模型的转换计划，包含一系列转换规则的集合，每种Plan代表一种转换结果。

- MagicPlanFilter

定义了一些正则表达式，在转换中，用于排除部分属性转换，优先级高于Rule，使用较少。

- MagicPlanRule

模型属性转换规则，是转换的核心，定义了每个属转换动作内容。

- MagicCode

> 码表映射定义，用于描述模型中码表属性的映射关系，通过映射关系快速转换码表。注意，如果Code关联了Plan，则表示是Plan专属Code，使用时优先于普通Code，如果Code未关联任何Paln，则表示是公用的。
>
> - MagicCodeItem
>
> 码表具体内容映射关系，码表值与值之间的映射，码表名+ItemValue（+ItemValue1，如果有）确定一个唯一映射关系。

规则载体

Json文件

当应用未依赖magic-resource时，规则默认在应用启动时通过扫描指定路径的Json文件获得，并保存在内存中，使用时通过planCode在MagicExplorer中查找。

文件扫描 Java

```java
public class MagicResourceRunner implements ApplicationRunner {
    @Resource
    private List<MagicResourceLoader> loaders;
    @Override
    public void run(ApplicationArguments args) throws Exception {
        Optional.ofNullable(loaders).ifPresent(loaders->loaders.forEach(MagicResourceLoader::load));
    }
}
public class PlanJsonResourceLoader extends JsonResourceLoader<MagicPlanBo> {
    @Value("${glory.magic.plan.resource.path:/magic/plan/*.json}")
    private String resourcePath;//扫描路径，一个Json文件表示一个Plan
    @Autowired
    private MagicExplorer magicExplorer;
    @Override
    protected void handleResource(MagicPlanBo resource) {
        magicExplorer.registerPlan( resource);
    }
    protected String getResourcePath(){
        return resourcePath;
    }
}
```

DB

如果项目依赖了magic-resource，那么模型会忽略Json而使用DB方式，下面介绍一下各数据表的核心字段。

- MagicPlan表

LIST_ID	integer *	Mock	plan id
PLAN_CODE	string *	Mock	plan code
ENABLED	integer *	Mock	status,1
NAME	string *	Mock	the plan name
KEY_NAME_STRATEGY	string N	Mock	result key name
EXCEPTION_STRATEGY	string N	Mock	exception
DESCRIPTION	string N	Mock	the plan
TENANT	string N	Mock	tenant code

- MagicPLanFilter表

LIST_ID	integer *	Mock	RULE id
PLAN_ID	integer *	Mock	plan id
NAME	string *	Mock	the rule name
ENABLED	integer *	Mock	status,1
DESCRIPTION	string N	Mock	the rule
TENANT	string N	Mock	tenant code
PATTERNS	string *	Mock	exclude

- MagicPlanRule表

LIST_ID	integer *	Mock	RULE id
PLAN_ID	integer *	Mock	RULE id
NAME	string *	Mock	the rule name
ALIAS	string N	Mock	the rule alias
ENABLED	integer *	Mock	status,1
TYPE	integer *	Mock	rule type,1
INPUT	string *	Mock	rule input,the
OUTPUT	string *	Mock	rule output,
PRIORITY	integer *	Mock	priority,sorted
FORMULA	string N	Mock	the rule formula
FORMAT	string N	Mock	the rule
STRATEGY	string *	Mock	the rule
VALUE	string N	Mock	rule default
CACHE_EVENT_KEY	string N	Mock	cache event key
SCALA	integer N	Mock	keep a certain
EXCLUDE_NODES	string N	Mock	when last digit
STEP_STRATEGY	string N *	Mock	the same
DESCRIPTION	string N	Mock	the rule
TENANT	string N	Mock	tenant code
PARAMETERS	string N	Mock	说明

- MagicCode表

LIST_ID	integer *	Mock	code id
PLAN_ID	integer N	Mock	plan id
ENABLED	integer *	Mock	status,1
NAME	string *	Mock	code table name
DESCRIPTION	string N	Mock	code description
TENANT	string N	Mock	tenant code

- MagicCodeItem表

LIST_ID	integer *	Mock	item id
CODE_ID	integer *	Mock	code table id
ENABLED	integer *	Mock	status,1
NAME	string *	Mock	the item name
DESCRIPTION	string N	Mock	the code item
TENANT	string N	Mock	tenant code
SRC_VALUE	string *	Mock	item src value
SRC_VALUE1	string N	Mock	item src value1
DEST_VALUE	string *	Mock	item dest

详细表设计可以参见magic-resource包下Liquibase的脚本。

组件设计

MagicEngine

MagicEngine 转换引擎 Java

```java
public interface MagicEngine {

    <T,R>MagicResponse<R> execute(@Nonnull MagicRequest<T> request);

    default <T,R>MagicResponse<R> execute(@Nonnull String planCode,@Nonnull T data){
        return execute(MagicRequest.create(planCode,data));
    }

    default <T,R>void executeAsync(@Nonnull MagicRequest<T> request,MagicCallback<MagicResponse<R>> callback){//异步
        CompletableFuture<MagicResponse<R>> future = CompletableFuture.supplyAsync(() -> execute(request), getExecutor());
        Optional.ofNullable(callback).ifPresent(c -> future.thenAccept(c::callback));
    }

    Executor getExecutor();//线程池
}
```

> 模型转换引擎，属于控制组件，是Magic暴露给外部调用的接口，支持同步和异步。

MagicProcessor

转换规则处理器 Java

```java
public interface MagicProcessor extends MagicAdapter {
    void process(@Nonnull MagicContext context);
}
```

> 规则处理器，属于控制组件，Node与Rule绑定后，在Processor中通过Rule的strategy找到具体的Stategy，并完成转换动作。

MagicFilter

MagicNode过滤器 Java

```java
public interface MagicFilter extends MagicAdapter {
    boolean doFilter(MagicContext context);
}
```

> 功能组件，在Normal Rule阶段过滤掉部分MagicNode，使用频率较小。

MagicStrategy

映射策略 Java

```java
public interface MagicStrategy extends MagicAdapter {
    //在ResultSet中产生一个空结果
    default ResultNode execute(MagicContext context){
        MagicPlanRuleBo rule = context.getRule();
        MagicNode node = context.getNode();
        ResultNode rn = context.getResult().addResultNode(rule.getReallyOutput(node));
        rn.setRule(rule);
        rn.setNode(node);
        Object value = node.getValue();
        rn.setValue(value);
        if (value == null){
            return rn;
        }
        return doExecute(new RuntimeMetadata(context, rn));
    }
    //具体映射策略
    default ResultNode doExecute(RuntimeMetadata metadata) {
        return metadata.getResultNode();
    }

    List<String> supportStrategies();//支持的策略名
}

```

> 功能组件，所有Rule的定义最终都会在Strategy中表现，每一种Strategy都表示一种转换动作，可根据需要扩展。

MagicHandler

补充任务处理器 Java

```java
public interface MagicHandler extends MagicAdapter {
    void handle(MagicContext context);
}
```

> 功能组件，补充处理器，用于处理一些特殊的转换需求，如GlobalRule就是在这里处理。

其它组件

- MagicResourceLoader

资源加载器 Java

```java
public interface MagicResourceLoader {
    void load();
}
```

> 功能组件，资源加载器，在应用启动时加载指定目录的资源文件。

- ExceptionHandler

异常处理器 Java

```java
public interface ExceptionHandler extends MagicAdapter {

    void handle(@Nonnull MagicException e);

    List<String> supportStrategies();//异常策略，默认panic
}
```

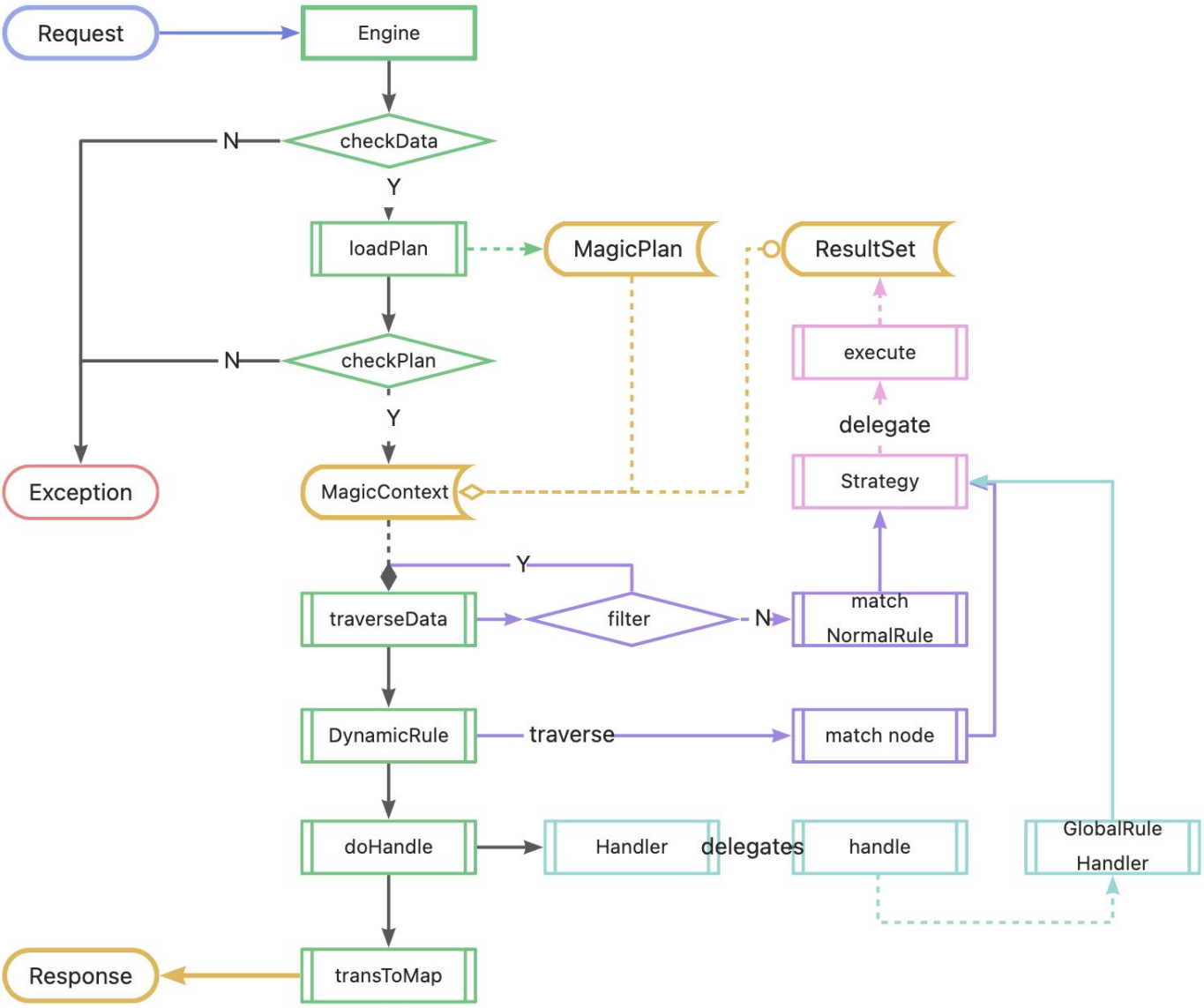
> 功能组件，用于处理模型转换过程中的异常。

流程设计

DefaultMagicEngine Java

```java
public class DefaultMagicEngine implements MagicEngine {

    @Override
    public <T,R> MagicResponse<R> execute(@Nonnull MagicRequest<T> request) {
        Assert.notNull(request.getData(), "Request data is null.");
        MagicPlanBo plan = request.getPlan();
        if (null == plan){//loadPlan，如果Plan不为空，优先使用request中指定的
            String planCode = request.getPlanCode();
            Assert.hasLength(planCode, "PlanCode is empty.");
            plan = resourceService.findPlan(planCode);
            Assert.notNull(plan, String.format("Plan [%s] is not found.", planCode));
        }
        MagicContext context = new MagicContext();//转换上下文
        context.setPlan(plan);//set plan
        context.initRequest(request);
        MagicResponse<R> response = new MagicResponse<>();
        response.setTenant(request.getTenant());
        response.setPlanCode(plan.getPlanCode());
        //遍历Data，Data先转换成Map然后遍历，这里对应NormalRule
        Consumer<MagicNode> consumer = node -> {//node叶子节点
            logger.trace(">> execute node: {}", node.getPath());
            doProcess(context.copyNode(node));
        };
        MagicNode rootNode = MagicHelper.traverse(request.getData(), consumer);
        //遍历DynamicRules
        BiConsumer<MagicNode, MagicPlanRuleBo> dynamicConsumer = (node, rule) -> {
            doProcess(context.copyDynamicRule(rule,node));
        };
        plan.processDynamicRules(rootNode, dynamicConsumer);
        context.setNode(rootNode);
        doHandle( context);//补充任务处理
        response.setResultMapData(context.getResult().transToMap(getKeyNameStrategy(request,plan)));//result to map
        return response;
    }
}
```



- checkData，校验Request中data（模型数据）是否为空，如果为空则异常关中断。
- loadPlan和check plan，如果Request中指定了MagicPlan，则跳过校验并优先使用，否则根据Request 的planCode查找MagicPlan，如未找到则异常并中断。
- MagicContext，模型转换的上下文对象，包含了转换所需要的数据，如MagicPlan，MagicNode、MagicPalnRule和ResultSet等，不同的阶段，内容也不一样。
- TraverseData，先将Reqeust中的data对象转换成Map对象，然后转换为MagicNode树，遍历过程中叶子节点会进入Processor流程：

> - filter ： 过滤器，主要用于排除部分MagicNode。
> - matchRule：通过Node匹配Rule，这里的Rule是NormalRule。
> - Strategy：根据Rule的strategy属性找到MagicStrategy，然后执行execute转换，并把结果保存到ResultSet中。

- DynamicRule：与TraverseData相反，这里是根据DynamicRule主动去匹配筛选Node，然后再Strategy流程。

- doHandle：补充任务，用于处理Normal和Dynamic Rule不方便处理的特殊需求任务。可顺序执行多个补充任务MagicHandler，GlobalRuleHandler便是其中一个，用于执行GlobalRule。
- transToMap：把ResultSet中保存的结果转换成Map，并保存到Response，Response中提供了把Map转换成指定类型的方法，可根据需要调用。

规则数据来源

遵循一个原则：Request > DB > 本地配置。***注意DB和本地配置是二先一关系***。

MagicPlan来源

- Request中包含了MagicPlan，优先级最高。
- DB，根据planCode从DB中查询，优先级次于Request，但高于本地配置文件。
- 本地配置文件，适用于没有依赖magic-resource包情况，根据planCode从MagicExplorer中查询，优先级最低。

MagicCode来源

- MagicPlan中带有，优先级最高。
- DB中查询获得，优先级次于MagicPlan。
- 本地配置文件，适用于没有依赖magic-resource包情况，优先级最低。

Strategy介绍

Strategy是MagicCube的核心功能组件，模型转换的具体动作就是在该组件中完成。

主要数据对象

MagicNode

模型数据Data在Traverse时把每个节点包装成MagicNode（文档内如未特别说明，Node就是指MagicNode），但只有叶子的Node会进入转换流程。Node中包含了当前节点name，path、value等信息，为便于查找当前层的其它节点内容，还缓存当前节点层为source（Map）。

ResultNode

目标数据模型的节点，保存于ResultSet中，转换过程中会根据Rule的output属性创建一棵结果树，ResultNode中包含了name、value、path等信息，以及转换时使用的Node和Rule。

> ResultNode创建时要注意output中是否包含了list结构：
>
> - 不包含
>
> 根据output创建ResultNode时，如果目标Node已经存在，则返回已存在的Node，如果未存在，则新建一个空ResultNode并返回。
>
> - 包含
>
> 根据output创建一个ResultNode时，每次都会新建一个ResultNode，因为List本身就是允许重复的。当output指向的叶子ResultNode存在时，会先把最近List的下标向后移，即新建一个List节点，然后重新创建一个ResultNode。这样做的一个充足理由是，当output中存在List结构时无法区分是要新建一个List节点，还是要返回已存在的节点，只能取舍。这也是Strategy拆分execute和doExecute二阶段执行的重要原因，有时只是想完成一个转换动作，但不要新创建一个ResultNode。

Rule的关键属性

- strategy：指定MagicStategy，名称匹配。
- input：MagicNode Path，Rule与Node的匹配通过该属性，支持限定正则（末端正则）。
- output：输出ResultNode Path，目标数据保存的位置，支持限定正则（末端正则）。
- type：RuleType，分别为Normal、Dynamic和Global三种：

> 1. Normal：被动匹配，在遍历Data时，由Node的path来匹配Rule的input。
> 2. Dyncmic：主动匹配，Data已经遍历完，MagicNode树构建完成，这时发起由Dynamic Rule的input去筛选MagicNodes。
> 3. Global：处理完简单转换数据后，Global用于处理复杂关系的数据转换。

接口说明

MagicStrategy Java

```java
public interface MagicStrategy extends MagicAdapter {

    default ResultNode execute(MagicContext context){
        MagicPlanRuleBo rule = context.getRule();
        MagicNode node = context.getNode();
        ResultNode rn = context.getResult().addResultNode(rule.getReallyOutput(node));
        rn.setRule(rule);
        rn.setNode(node);
        Object value = node.getValue();
        rn.setValue(value);
        if (value == null){
            return rn;
        }
        return doExecute(new RuntimeMetadata(context, rn));
    }

    default ResultNode doExecute(RuntimeMetadata metadata) {
        return metadata.getResultNode();
    }

    List<String> supportStrategies();//指定Strategy名，与Rule的strategy匹配
}
```

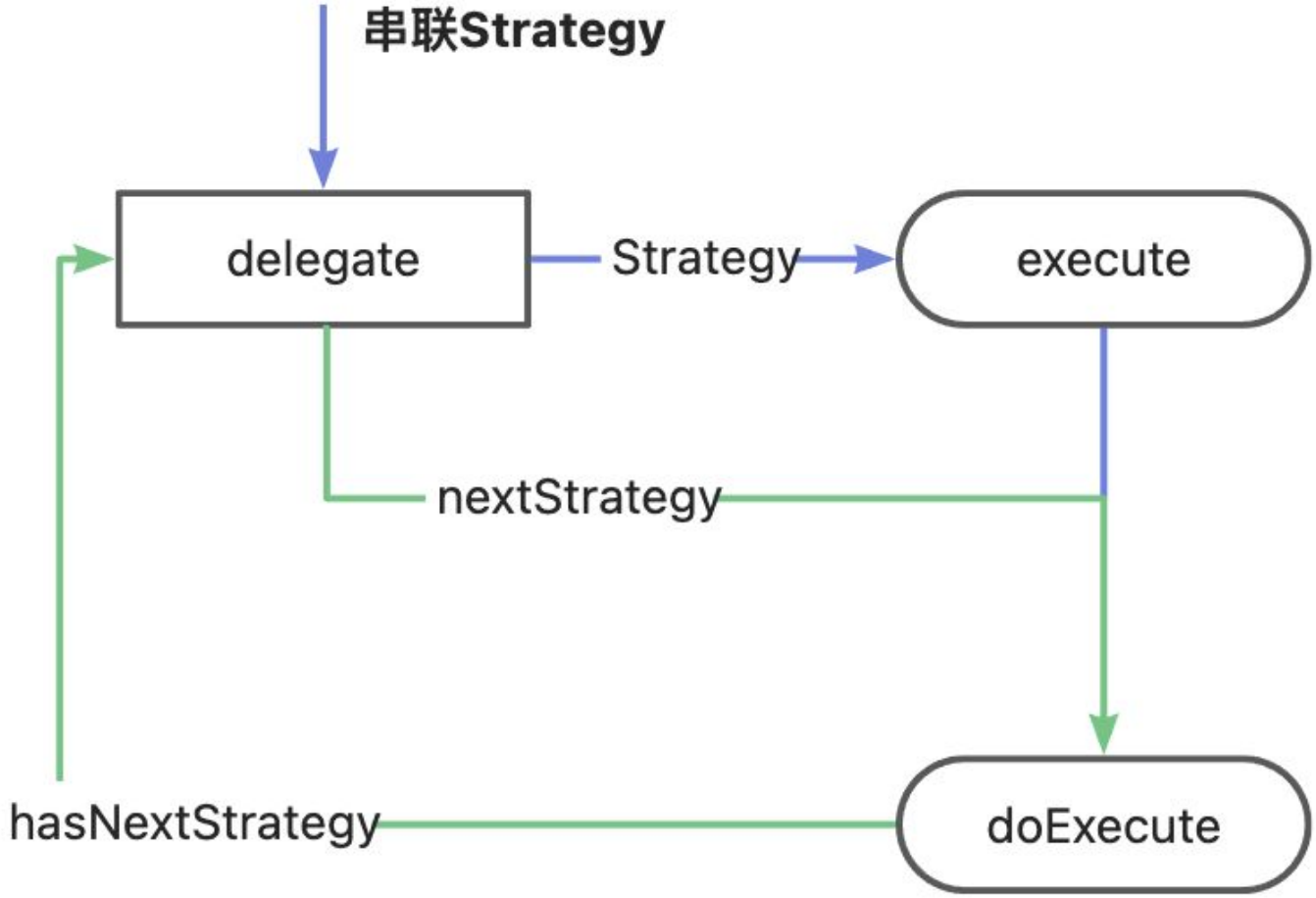
> 设计中，因MagicStrateg继承MagicAdapter，所以支持planCode条件过滤，也就是说Strategy可以是某个Plan私有，如未指定，则表示是公有的。在业务处理中，考虑到某些复杂动作在一个Strategy中完成困难较大，可组成串联模式完成复杂动作，提高Strategy的复用率。

execute

execute是动作入口，主要职责是在ResultSet中根据Rule的output生成一个空的ResultNode，如果当前MagicNode的value是空，则直接返回并路过doExecute环节。

doExecute

是Strategy具体动作的执行环节，也是串联模式时的动作入口，建议Strategy的具体业务动作在该方法中实现。在Strategy串联模式中，除第一个Strategy是从execute进入，后续的Strategy都是直接执行doExecute，串联顺序通过Rule的stepStrategy指定顺序。



常用Strategy

valueCopy

使用最广泛的策略之一，功能简单，就是把input的MagicNode的value拷贝到output指定的的ResultNode中。

fixedValue

固定值，输出节点ResultNode的value来自Rule中定义的value值，常用于向ResultSet中插入一个常量值。

codeMap

码表映射策略，用于处理模型中的码表转换，码表映射需要提前准备，根据codeName和value找到目标value。

- format：指定码表名

> 附加属性
>
> 1. srcValue1Key：如果有，则表示码表映射的第二value。

dateFormat

日期格式化策略，用于日期类型数据格式化输出。

- format：指定日期输出格式，如：yyyy/MM/dd HH:mm:ss。

> 附加属性
>
> 1. dateFormat：当获取的源数据值是字符串类型时，需要根据dateFormat转换成日期对象。

numberFormat

数值格式化策略，用于数字类型格式化输出。

- format：指定数据输出格式，如#,###.00，表示千分位，并保留2位有效小数。

> 附加属性
>
> 1. formatType：枚举：0，数值；1，货币；2，百分比

stringFormat

字符串格式策略，用于字符串格式化输出。

- format：表示格式化类型

1. upper：大写
2. lower：小写
3. first：首字母大写
4. mask：掩码处理，

mask Rule示例 JSON

```
{
    "name":"master",
    "input":"master#emails",
    "output":"holder#emailMask2",
    "format":"mask",
    "pattern":"^(.{4}).*(.{4})$",
    "replacement":"$1****$2",
    "minLength":"14",
    "minLengthStr":"****",
    "strategy": "stringFormat"
}
//input: tianditong@163.com |output: tian****.com
```

- 附加属性：

1. pattern：正则表达式，匹配模板，与replacemen配合使用，例如：^(.{4}).*$}。
2. replacement：替换模板，$1****，表示保留前4位不变，后面*补位。
3. minLength：字符串模板匹配最小长度，当小于这个长度，则默认为minLengthStr的值。
4. minLengthStr：小于最小长度时，默认值。

5. encrypt：加密

encrypt DES示例 JSON

```
{
  "name":"master",
  "input":"master#email",
  "output":"holder#emailDES",
  "algorithm":"DES",
  "format":"encrypt",
  "strategy": "stringFormat"
}
//input: tianditong@163.com |output: gks4nrnvePvgL+CKRtcnk7NceAjFLzIi
```

附加属性：

1. algorithm，指定加密方式：AES、DES、RSA和BASE64.

publishCacheEvent

相当于一个延时策略，先把当前上下文数据缓存到ResultSet中，等consumeCacheEvent策略来唤醒继续执行，常与其他策略组成串联完成转换动作。该策略主要解决在顺序遍历过程中，前面节点依赖后续

节点的创建，所以需要暂缓策略动作执行，等后续关键节点执行时再来唤醒执行。

注意，策略本身不执行任何动作，只会根据ouput创建ResultNode，具体的动作常在后续的串联Strategy中完成。

- cacheEventKey：缓存Key，用于标识一组缓存，一个key可以包含多个缓存。
- stepStrategy：指定串联动作的Strategy。

consumeCacheEvent

策略本身没有转换动作，只是单一作为publishCacheEvent的唤醒策略。

- cacheEventKey：缓存Key，用于标识一组缓存，与publishCacheEvent通过Key关联。

groovyFormula

全局公式计算策略，主要用于处理需要公式计算的动作，如对模型内某些数据进行统计等。

groovyFormula Rule示例 JSON

```
{
  "name": "global",
  "output": "totalAmount",//输出结果path
  "formula": "(butie+salary)*(1-tax)*member+ms*freq+jiangjin*0.8",//公式
  "parameters":{//公式参数列表
    "$CA_salary":"salarys_cache",//$CA_表头，表示一组缓存的和，后面是参数名
    "$R_butie":"butie",//$R_表示从根节点开始取值，后面参数名
    "$R_tax":"tax",
    "$R_member":"member",
    "$R_ms":"master#salary",
    "$R_freq":"master#freq",
    "jiangjin":10000//parameters中设置的常量
  },
  "scala": 2,//小数位
  "type":2,//global rule
  "strategy": "groovyFormula"
}
```

- formaula：计算公式。
- scala：结果保存小数位。
- parameters：公式参数列表，具体如下：

> 1. $CA_XXX
>
> - $CA_：表示一组通过publishCacheEvent设置的缓存，并取和

- XXX：参数名，公式中的变量名。
- value：publishCacheEvent的cacheEventKey。

2. $C_XXX

- $C_：表示一组通过publishCacheEvent设置的缓存，但是只取其中一个，如果value中没有指定，则取第一个。
- XXX：参数名，公式中的变量名。
- value：NodePath{;CacheNodePath}，CacheNodePath是可选的，未指定时，表示取第一个。如果指定，则与CacheEvent中Node的Path匹配选取。

3. $N_XXX

- $N_：表示从同及节点中取舍，即在MagicNode的source中取。
- XXX：参数名，公式中的变量名。
- value：属性名。

4. $R_XXX

- $R_：表示从根节点根据path取值。
- XXX：参数名，公式中的变量名。
- value：从根节点开始的NodePath路径。

5. 无前缀XXX

- XXX：参数名，公式中的变量名。
- value：常量。

自定义Strategy

在模型转换中，存在很多想不到的场景，上述几个Strategy只是常见的几种场景。面对问题时还需要具体问题具体分析，根据实际需求自定义扩展MagicStrategy。

Rule路径

- input：必需，既是数据来源，也是触发发条，即与MagicNode的path是否匹配成功。
- output：必需，转换结果的输出path，也是ResultNode的path。
- input和output路径规则

  - #：表示层级分隔，address#country表示address下的country属性。
  - -*：表示当前层list结构，*表示下标，在Rule中表示List下标时，尽量不要用具体数字，用*来动态匹配。如members-*#friends-*#pay，members和friends都是List结构。

- 末端*：表示末端模糊匹配，如house#*，表示house下的所有属性。

测试

相关API

核心API

- 查看码表

URI：/core/viewCode/{codeName}

Method：GET

codeName：MagicCode唯一标识符

- 查看MagicPlan

URI：/core/viewPlan/{planCode}

Method：GET

planCode：MagicPlan的唯一标识符。

- 模型转换

URI：/core/execute

Method：POST

RequestBody：Json

execute RequestBody JSON

```
{
    "planCode": "planCode",//与plan不能同时为空
    "plan": {},//优先级高于planCode，与planCode不能同时为空
    "withDefaultRule": true,//示匹配到Rule的节点，默认valueCopy strategy Rule
    "tenant":"tenantCode"//租户Code，可选
    "exceptionStrategy":"tryCache"//异常处理策略，默认tryCache
    "keyNameStrategy": "normalStrategy",//Result map key命名策略，默认Normal
    "data": {}//必须，模型转换数据
}
```

- planCode和plan不能同时为空，并存时plan优先于planCode。plan通常用于临时转换或是测试plan的准确性，正常推荐使用planCode。

- withDefaultRule，默认为false，当MagicNode未匹配到NormalRule时，是否默认匹配一个valueCopy Rule，适用于模型只需要做少量调整，其它保持不变的情况，这样只需要Rule配置即可。
- exceptionStrategy：异常处理策略，转换过程出现异常如何处理，如果plan中也未设置策略，则默认tryCache。
- keyNameStrategy，Result转换成Map时，Key的命名策略：firstStrategy，upperStrategy，lowerStrategy，normalStrategy。默认normalStrategy不做处理，可根据需要扩展ResultKeyNameStrategy接口。

资源类API

资源API主要是与MagicPlan和MagicCode相关的增、删、改、查操作，由于API较多，这里不一一介绍，详细可以参见Magic API.html文件。

测试

测试数据

Case 测试数据Data JSON


```
{
  "name": "faimly1",
  "member": 4,
  "butie":456.0,
  "tax":0.2,
  "house": {
    "name": "heping",
    "category": "minfang",
    "area": 90.0,
    "buildTime": "2000-03-01T00:00:00"
  },
  "master": {
    "name": "zhigao",
    "age": 50,
    "sex": "M",
    "role": "jiazhu",
    "birthday": "1970-03-11T00:00:00",
    "email": "dsfsfs@163.com",
    "phone": "9233242",
    "work": "dev",
    "salary": 10000.00
  },
  "members": [
    {
      "name": "zhigao",
      "age": 50,
      "sex": "M",
      "role": "jiazhu",
      "birthday": "1970-03-11T00:00:00",
      "email": "dsfsfs@163.com",
      "phone": "9233242",
      "work": "dev",
      "salary": 10000.00,
      "friends": [
        {
          "name": "hebi",
          "age": 45,
          "sex": "M",
          "phone": "3455646464"
        },
        {
          "name": "nancy",
          "age": 60,
          "sex": "W",
          "phone": "89978ww9"
        }
      ]
    },
    {
      "name": "tiantian",
      "age": 48,
      "sex": "W",
      "role": "wife",
```

```
      "birthday": "1977-03-11T00:00:00",
      "email": "sdfsa@163.com",
      "phone": "345645"
    },
    {
      "name": "wawa",
      "age": 18,
      "sex": "W",
      "role": "girl",
      "birthday": "1999-05-11T00:00:00",
      "email": "wawa@163.com",
      "phone": "9999999",
      "friends": [
        {
          "name": "jarrod",
          "age": 20,
          "sex": "W",
          "phone": "0999999"
        },
        {
          "name": "yongwei",
          "age": 23,
          "sex": "M",
          "phone": "888888888"
        },
        {
          "name": "limi",
          "age": 12,
          "sex": "M",
          "phone": "444444"
        }
      ]
    },
    {
      "name": "gege",
      "age": 28,
      "sex": "M",
      "role": "sun",
      "birthday": "1990-05-11T00:00:00",
      "email": "gege@163.com",
      "phone": "33333"
    }
  ],
  "pets": [
    {
      "name": "dahuang",
      "category": "dog",
      "age": 3,
      "weight": 20
    }
  ],
  "address": {
    "country": "zh",
    "province": "sh",
    "city": "pudong",
    "street": "chengshan rold"
```

```
111      }
       }
```

> 如未特别指定，后续测试Case使用的数据Data均为该数据。

测试Case

valueCopy使用

Request Rules JSON

```
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "valueCopy",
        "rules": [
            {//同层级拷贝，末断*号，表示匹配同一层级所有属性，属性名同打他节点名。
                "name": "master",
                "input": "master#*",//输入
                "excludeNodes": ["work","sex"],//排除内容
                "output": "master#*",//输出
                "strategy": "valueCopy"
            },
            {//单节点拷贝
                "name": "address",
                "input": "address#country",//输入
                "output": "newAdd#test",//输出
                "strategy": "valueCopy"
            },
            {//固定值策略
                "name": "address",
                "input": "address#city",//触发输入
                "output": "newAdd#wwww",//输出
                "value": "waaa",//固定值
                "strategy": "fixedValue"
            }
        ]
    },
    "withDefaultRule": false
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "valueCopy",
    "resultMapData": {
        "newAdd": {
            "test": "zh",
            "wwww": "waaa"
        },
        "master": {
            "birthday": "1970-03-11T00:00:00",
            "role": "jiazhu",
            "phone": "9233242",
            "name": "zhigao",
            "salary": 10000.0,
            "age": 50,
            "email": "dsfsfs@163.com"
        }
    }
}
```


> 本Case演示了valueCopy和fixedValue两种Strategy的使用方法，重点是valueCopy，使用频率最高的Strategy之一，单节点匹配、同层级匹配及属性排除的用法。

List条件拆分

如何把一个List根据某个条件拆分几个小List。

List条件拆分Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "splitList",
        "rules": [
            {
                "name":"members",
                "input":"members-*#friends-*#*",
                "output":"pals#${name}-*#*",//${name}表示一个变量，name是变量名
                "excludeNodes":["pay"],
                "strategy": "valueCopy",
                "nameKey":"sex",//折分条件sex
                "name_M":"mans",//分组条件sex=M，组名mans
                "name_W":"womens"//分组条件sex=W，组名womens
            },
            {
                "name":"members",
                "input":"members-*#friends-*#pay",
                "output":"pals#${name}-*#pay",
                "strategy": "valueCopy",
                "nameKey":"sex",
                "name_M":"mans",
                "name_W":"womens",
                "format":"payMod",
                "stepStrategy":["codeMap"]
            }
        ],
        "codes":[
            {
                "name":"payMod",
                "items":[
                    {
                        "name":"aliPay",
                        "srcValue":1,
                        "destValue":"A"
                    },
                    {
                        "name":"wePay",
                        "srcValue":2,
                        "destValue":"W"
                    },
                    {
                        "name":"cash",
                        "srcValue":3,
                        "destValue":"C"
                    }
                ]
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "splitList",
    "resultMapData": {
        "pals": {
            "mans": [
                {
                    "phone": "3455646464",
                    "sex": "M",
                    "name": "hebi",
                    "age": 45
                },
                {
                    "phone": "888888888",
                    "sex": "M",
                    "name": "yongwei",
                    "age": 23
                },
                {
                    "phone": "444444",
                    "sex": "M",
                    "name": "limi",
                    "age": 12
                }
            ],
            "womens": [
                {
                    "phone": "89978ww9",
                    "sex": "W",
                    "name": "nancy",
                    "age": 60
                },
                {
                    "phone": "0999999",
                    "sex": "W",
                    "name": "jarrod",
                    "age": 20
                }
            ]
        }
    }
}
```


Case是根据sex=X来把members-*#friends-*#*拆分成mans和womens两个List。

- output【pals#${name}-*#*】中定义了一个变量name，用不确定的变量来实现动态分组。
- 分组条件：nameKey=sex，格式：变量名+Key=分组属性名，value分组条件属性名，这里指定sex为分组属性。
- 分组值：name_M=mans和name_W=womens是两个分组，格式：变量名+条件Value=分组名，

> value=分组名，如name_M=mans，表示sex=M的分到mans一组中。

分组条件建议选择可枚举的属性。

动态Rule Case

动DynamicRule与NormalRule的最大区别在于与MagicNode匹配的方式，Normal是被动匹配，而Dynamic是主动匹配。

Dynamic Rule JSON

```
{
    "planCode": "planCode",
    "plan": {
        "name": "family dynamic",
        "planCode": "dynamicRule",
        "rules": [
            {
                "input": "master#*",
                "name": "test",
                "output": "master#*",
                "strategy": "valueCopy",
                "type": 1//Dynamic Rule, 默认0, normalRule
            },
            {
                "input": "pets-*#*",
                "name": "test",
                "output": "master#pets-*#*",
                "strategy": "valueCopy",
                "type": 1//Dynamic Rule
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "dynamicRule",
    "resultMapData": {
        "master": {
            "birthday": "1970-03-11T00:00:00",
            "pets": [
                {
                    "name": "dahuang",
                    "weight": 20,
                    "category": "dog",
                    "age": 3
                }
            ],
            "role": "jiazhu",
            "phone": "9233242",
            "work": "dev",
            "sex": "M",
            "name": "zhigao",
            "salary": 10000.0,
            "age": 50,
            "email": "dsfsfs@163.com"
        }
    }
}
```


> Case介绍了动态Rule的配置，即type=1表示动态Rule。

字符串格式化Case

stringFormat Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "stringFormat",
        "rules": [
            {
                "name":"master",
                "input":"master#*",
                "output":"holder#*",
                "excludeNodes":["email"],
                "strategy": "valueCopy"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailUpper",
                "format":"upper",//大写
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailLower",
                "format":"lower",//小写
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailFirst",
                "format":"first",//首字母大写
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailMask",
                "format":"mask",//掩码
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailMask1",
                "format":"mask",//自定义掩码格式
                "pattern":"^(.{4}).*(.{4})$",//匹配模板
                "replacement":"$1****$2",//替换模板
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#emails",
```

```
                "output":"holder#emailMask2",
                "format":"mask",//自定义掩码格式
                "pattern":"^(.{4}).*(.{4})$",//匹配模板
                "replacement":"$1****$2",//替换模板
                "minLength":"14",//匹配小时长度
                "minLengthStr":"****",//小于最小长度时返回默认值
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailAES",
                "format":"encrypt",//默认AES加密
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailDES",
                "algorithm":"DES",//DES加密
                "format":"encrypt",//加密
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailRSA",
                "algorithm":"RSA",//RSA加密
                "format":"encrypt",//加密
                "strategy": "stringFormat"
            },
            {
                "name":"master",
                "input":"master#email",
                "output":"holder#emailBase",
                "algorithm":"BASE64",//Base64加密
                "format":"encrypt",//加密
                "strategy": "stringFormat"
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "stringFormat",
    "resultMapData": {
        "holder": {
            "birthday": "1970-03-11T00:00:00",
            "emailFirst": "Tianditong@163.com",
            "role": "jiazhu",
            "work": "dev",
            "sex": "M",
            "emailDES": "gks4nrnvePvgL+CKRtcnk7NceAjFLzIi",
            "emailUpper": "TIANDITONG@163.COM",
            "salary": 10000.0,
            "emailLower": "tianditong@163.com",
            "emailAES": "oPeGOupUVMaYMReAZDhHUuq0QvQ4OgW/6tYlfMmPRyE=",
            "emails": "sort@163.com",
            "phone": "17022229999",
            "emailMask": "tian****",
            "emailBase": "dGlhbmRpdG9uZ0AxNjMuY29t",
            "name": "zhigao",
            "emailRSA": "eCgYR4qQYswgDsl1vNYxhmaiM6rwUdmSHRkILWbN3+ebdtilxuzB+zrXtZFe1MfcZY2ap4w98tl1hXYbnzWsGcCqwKJ3fEMR1/WwaVqkI2+100wLHiQMMHo+f1bYbQOYdP095t0j70rSO22fB0ey1em1uzQP9m76QJS77BrxnL2/tlaNWOWVnvz5qCYNDWWW+K+pPCwEKvBr+Xhkrofq1bcwv9Y1IQz6mRoSNuN1F12blutSx28+xtvaWjdeybX7v1h8wqd38xv+pM1Wuc6WnZBi5pWjgPFe4OHg4Cl4aXTlAIHK/wzo0RaOf/TEXfVOTcc0RamsIheF4dAtY2MJkQ==",
            "emailMask2": "****",
            "emailMask1": "tian****.com",
            "age": 50
        }
    }
}
```


> Case介绍了stringFormat的多种使用方式。

数值格式化Case

numberFormat Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "numberFormat",
        "rules": [
            {
                "name":"master",
                "input":"master#*",
                "output":"holder#*",
                "excludeNodes":["salary"],
                "strategy": "valueCopy"
            },
            {
                "name":"master",
                "input":"master#salary",
                "output":"holder#salary",
                "formatType":"0",//数值
                "format":"#,###.0000",//4位小数
                "strategy": "numberFormat"
            },
            {
                "name":"master",
                "input":"master#salary",
                "output":"holder#salary1",
                "formatType":"0",//数值
                "format":"###.0",//1位小数
                "strategy": "numberFormat"
            },
            {
                "name":"master",
                "input":"master#salary",
                "output":"holder#salaryCurrency",
                "formatType":"1",//货币
                "format":"$#,###.00",//2位小数
                "strategy": "numberFormat"
            },
            {
                "name":"master",
                "input":"master#salary",
                "output":"holder#salaryCurrency1",
                "formatType":"1",//货币
                "format":"RMB:#,###.00",//带前缀
                "strategy": "numberFormat"
            },
            {
                "name":"master",
                "input":"master#salary",
                "output":"holder#salary3",
                "formatType":"2",//百分数
                "format":"#,###.00",
                "strategy": "numberFormat"
            }
```

```
            ]
        }
    }
```

Result JSON


```
{
    "status": 1,
    "planCode": "numberFormat",
    "resultMapData": {
        "holder": {
            "birthday": "1970-03-11T00:00:00",
            "role": "jiazhu",
            "work": "dev",
            "sex": "M",
            "salaryCurrency": "$244,233,234.57",
            "salary": "244,233,234.5670",
            "salaryCurrency1": "RMB:244,233,234.57",
            "phone": "17022229999",
            "salary1": "244233234.6",
            "name": "zhigao",
            "salary3": "24,423,323,457%",
            "age": 50,
            "email": "tianditong@163.com"
        }
    }
}
```


> Case介绍了numberFormat的几种使用方式。

codeMap Case

这里的码表是指模型转换中，同一码表不同定义间的映射关系，如同一CodeTable在模型A中的值为A，但是在模型B中的值可能是1，为方便码表转换，需要做好模型A到模型B间的码表值映射。

codeMap Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "codeMap",
        "rules": [
            {
                "name": "master",
                "input": "master#*",
                "output": "holder#*",
                "excludeNodes": ["sex"],
                "strategy": "valueCopy"
            },
            {
                "name": "master",
                "input": "master#sex",//码表值
                "output": "holder#sex",
                "format": "gender",//码表名
                "strategy": "codeMap"
            }
        ],
        "codes": [//码表映射关系
            {
                "name": "gender",//码表名
                "items": [
                    {
                        "name": "woman",
                        "srcValue": "W",//码表值
                        "destValue": "F"//转换值
                    },
                    {
                        "name": "man",
                        "srcValue": "M",
                        "destValue": "B"
                    }
                ]
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "codeMap",
    "resultMapData": {
        "holder": {
            "birthday": "1970-03-11T00:00:00",
            "role": "jiazhu",
            "phone": "17022229999",
            "work": "dev",
            "sex": "B",
            "name": "zhigao",
            "salary": 10000.0,
            "age": 50,
            "email": "tianditong@163.com"
        }
    }
}
```


> Case是把master#sex的值按码表gender描述的映射关系转换成目标值再输出。

stepStrategy应用Case

有些情况，一个属性的转换需要多个动作协同才能完成，stepStrategy串联便是解决这类问题的一个解决方案，多个Strategy串联只有第一个Strategy会产生结果，后续的Strategy只会对前面的结果进一步调整。

stepStrategy Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "stepStrategy",
        "rules": [
            {
                "name": "master",
                "input": "master#*",
                "output": "holder#*",
                "strategy": "valueCopy"
            },
            {
                "name": "master",
                "input": "master#sex",
                "output": "holder#car",//输出
                "value": "B",
                 "format": "carType",//码表名
                "strategy": "fixedValue",//固定值策略，先向结果中插入一个值car=B
                "stepStrategy":["codeMap"]//然后码表转换B->BMO
            }
        ],
        "codes": [
            {
                "name": "carType",
                "items": [
                    {
                        "name": "baoma",
                        "srcValue": "B",
                        "destValue": "BMO"
                    },
                    {
                        "name": "fute",
                        "srcValue": "F",
                        "destValue": "FT"
                    }
                ]
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "stepStrategy",
    "resultMapData": {
        "holder": {
            "birthday": "1970-03-11T00:00:00",
            "role": "jiazhu",
            "car": "BMO",//结果
            "phone": "17022229999",
            "work": "dev",
            "sex": "M",
            "name": "zhigao",
            "salary": 10000.0,
            "age": 50,
            "email": "tianditong@163.com"
        }
    }
}
```


> 本Case结合了fixedValue和codeMap策略的串联应用，先在输出结果中插入了car=B，然后串联Strategy codeMap把B转换为BMO。。

KeyNameStategy Case

在实际需求中，不同模型对属性名的命名也可能存在差别，大多数会使用默认变量命名，有些模型则会要求首字母大写，或是大写等要求，这里提供了一种解决方案。

keyNameStrategy Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "keyNameStrategy",
        "rules": [
            {
                "name": "master",
                "input": "master#*",
                "output": "holder#*",
                "strategy": "valueCopy"
            },
            {
                "name": "master",
                "input": "master#sex",
                "output": "holder#car",//输出
                "value": "B",
                 "format": "carType",//码表名
                "strategy": "fixedValue",//固定值策略，先向结果中插入一个值car=B
                "stepStrategy":["codeMap"]//然后码表转换B->BMO
            }
        ],
        "codes": [
            {
                "name": "carType",
                "items": [
                    {
                        "name": "baoma",
                        "srcValue": "B",
                        "destValue": "BMO"
                    },
                    {
                        "name": "fute",
                        "srcValue": "F",
                        "destValue": "FT"
                    }
                ]
            }
        ]
    },
  "keyNameStrategy": "upperStrategy"//key 大写
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "keyNameStrategy",
    "resultMapData": {
        "HOLDER": {
            "ROLE": "jiazhu",
            "SALARY": 10000.0,
            "CAR": "BMO",
            "PHONE": "17022229999",
            "WORK": "dev",
            "EMAIL": "tianditong@163.com",
            "BIRTHDAY": "1970-03-11T00:00:00",
            "NAME": "zhigao",
            "AGE": 50
        }
    }
}
```


> Rule的配置与stepStrategy几乎一样，只是多了一个 keyNameStrategy=upperStrategy，输出结果的Key却完全不同。

缓存与公式计算Case

缓存与公式计算Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "groovyFormula",
        "rules": [
            {
                "name": "butie",
                "input": "butie",
                "output": "butie",
                "cacheEventKey":"salary_cache",//把当前上下文缓存
                "strategy": "publishCacheEvent"
            },
            {
                "name": "master",
                "input": "master#*",
                "output": "holder#*",
                "excludeNodes": ["salary"],
                "strategy": "valueCopy"
            },
            {
                "name": "master",
                "input": "master#salary",
                "output": "holder#salary",
                "formula": "(butie+salary)*freq",//计算公式
                "parameters":{
                    "$N_freq":"freq",//freq参数从当前层中获取
                    "$C_butie":"salary_cache;butie"//butie参数从缓存中获取
                },
                "scala": 2,
                "strategy": "groovyFormula"
            }
        ]
    }
}
```

Result JSON


```
{
    "status": 1,
    "planCode": "groovyFormula",
    "resultMapData": {
        "holder": {
            "birthday": "1970-03-11T00:00:00",
            "role": "jiazhu",
            "phone": "17022229999",
            "work": "dev",
            "sex": "M",
            "name": "zhigao",
            "freq": 12,
            "salary": 125472.00,//计算结果
            "age": 50,
            "email": "tianditong@163.com"
        }
    }
}
```


> 注意参数的格式，前面groovyFormula章节有详细说明。

GlobalRule和公式计息Case

GlobalRule和公式计息Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "groovyFormula",
        "rules": [
            {
                "name": "members",
                "input": "members-*#salary",
                "output": "members-*#salary",
                "cacheEventKey":"salarys_cache",//缓存
                "strategy": "publishCacheEvent"
            },{
                "name": "global",
                "output": "totalAmount",
                "formula": "(butie+salary)*(1-tax)*member+ms*freq+jiangjin*0.8",
                "parameters":{
                    "$CA_salary":"salarys_cache",//salarys_cache缓存之和
                    "$R_butie":"butie",//从根节点开始取值
                    "$R_tax":"tax",
                    "$R_member":"member",
                    "$R_ms":"master#salary",
                    "$R_freq":"master#freq",
                    "jiangjin":10000
                },
                "scala": 2,//小数位
                "type":2,//Global Rule
                "strategy": "groovyFormula"
            }
        ]
    }
```

Result JSON

```json
{
    "status": 1,
    "planCode": "groovyFormula",
    "resultMapData": {
        "totalAmount": 257459.20
    }
}
```

Case是一个涉及全局性数据的计息，所以需要注意

- 计息的时机，计算时要考虑所需要的参数是否已经就位，如缓存是否已经存在，data是否已经完全转换为MagicNode树结构。
- 参数来源多样性，根据注意参数的前缀标识来区分。

CacheEvent use Case

CacheEvent Case JSON

```json
{
    "planCode": "planCode",
    "plan": {
        "name": "test",
        "planCode": "consumeCacheEvent",
        "rules": [
            {
                "name": "butie",
                "input": "butie",
                "output": "holders-*#butie",
                "cacheEventKey":"salary_cache",
                "strategy": "publishCacheEvent",
                "format":"$#,###.000",
                "stepStrategy":["numberFormat"]
            },
            {
                "name": "members",
                "input": "members-*#*",
                "output": "holders-*#*",
                "strategy": "valueCopy"
            },{
                "name": "members",
                "input": "members-*#phone",
                "output": "holders-*#butie",
                "cacheEventKey":"salary_cache",
                "strategy": "consumeCacheEvent"
            }
        ]
    }
```

Result JSON


```
{
    "status": 1,
    "planCode": "consumeCacheEvent",
    "resultMapData": {
        "holders": [
            {
                "birthday": "1970-03-11T00:00:00",
                "role": "jiazhu",
                "phone": "9233242",
                "butie": "$456.000",//
                "work": "dev",
                "sex": "M",
                "name": "zhigao",
                "salary": 10000.0,
                "age": 50,
                "email": "dsfsfs@163.com"
            },
            {
                "birthday": "1977-03-11T00:00:00",
                "role": "wife",
                "phone": "345645",
                "butie": "$456.000",//
                "sex": "W",
                "name": "tiantian",
                "salary": 10000.0,
                "age": 48,
                "email": "sdfsa@163.com"
            },
            {
                "birthday": "1999-05-11T00:00:00",
                "role": "girl",
                "phone": "9999999",
                "butie": "$456.000",//
                "sex": "W",
                "name": "wawa",
                "salary": 10000.0,
                "age": 18,
                "email": "wawa@163.com"
            },
            {
                "birthday": "1990-05-11T00:00:00",
                "role": "sun",
                "phone": "33333",
                "butie": "$456.000",//
                "sex": "M",
                "name": "gege",
                "salary": 10000.0,
                "age": 28,
                "email": "gege@163.com"
            }
        ]
    }
}
```

> 上述Case是把根结点的butie属性分别插入到各个members节点下，难点：
>
> - 遍历顺序问题，在butie节点时，members节点还没有创建，无法操作，只能暂缓执行，等members节点创建再来唤醒执行。
> - 一对多问题，一个补贴要插入4个members节点中，members是一个List结构，每次output创建结果时，都会产生一个新的ResultNode，所以butie的插入要与members valueCopy同步进行。

特别建议

测试Case中，MagicPlan都是在Request中指定，原因：

- 首先是方便Case介绍，放在Requst请求中，可直观看到测试Rule的内容。
- 其次为了方便调试Rule的配置，配置Rule时可随时调整Rule的内容，以快速达到预期结果。

但在实际应用中，建议把已调试好的MagicPlan保存到本地文件中或是存入DB中，Request中只需要指定planCode即可，方便后续的管理和维护。